

ECOM EXPERTS

Health Corner

Manual de usuario e instalacion

ecom_healthcorner_css 1.0.0 · 04/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

Multitienda — léase antes de instalar

3. Instalación

Qué crea al instalarse

Qué pasa al desinstalar

4. Configuración

Pestaña «Pago (Redsys)»

Pestaña «Checkout y emails»

Pestaña «Panel de negocios»

Pestaña «Máquinas y planograma»

Pestaña «API»

Pestaña «Aislamiento»

Pestaña «Avanzado»

5. Cómo se usa, día a día

Dar de alta un hotel

Poner productos y precios

El día a día del hotel

El reponedor

Liquidaciones

Conectar la máquina expendedora

La tarea programada

6. Qué ve el cliente en la tienda

7. Preguntas frecuentes y problemas

8. Seguridad

Lo que ya estaba bien

Lo que se ha corregido en esta revisión

Comprobaciones hechas sobre los cambios

Lo que queda pendiente

9. Ficha técnica

Identificación

Tablas

Hooks

Controladores de administración

Direcciones del front

API

Tarea programada

Registros

Ajustes guardados

Permisos de ficheros

Documentación técnica adicional

1. Qué hace y para quién

Health Corner convierte una tienda PrestaShop en una red de **tiendas de parafarmacia independientes, una por hotel**. Cada establecimiento tiene su propia dirección web, su propio logotipo, su color, su catálogo, sus precios y sus hojas legales. El huésped entra escaneando un código QR que hay en recepción, en la habitación o en la propia máquina expendedora, y se encuentra una tienda que parece del hotel, no de un tercero.

El problema que resuelve es que un hotel no quiere montar un comercio electrónico y un distribuidor de parafarmacia no quiere montar una tienda por cada hotel al que sirve. Con este módulo hay **una sola tienda PrestaShop** —un solo catálogo, un solo almacén, una sola contabilidad— y encima de ella tantos escaparates como establecimientos haya. Dar de alta un hotel nuevo es rellenar una ficha: el módulo le genera la dirección, el código QR, sus páginas legales y su panel de gestión.

El huésped no se registra ni crea una cuenta. Compra como invitado indicando su nombre y su número de habitación, paga con tarjeta, y recibe un **código de recogida** de ocho caracteres. Ese código es la prueba: lo enseña en recepción si el pedido se le entrega en mano, o lo teclea en la máquina expendedora si lo recoge allí. Una página de seguimiento le muestra en todo momento en qué punto está su pedido, sin que tenga que llamar a nadie.

Hay tres personas trabajando alrededor del módulo, y cada una tiene su pantalla. El **personal del hotel** entra en un panel propio donde ve los pedidos pendientes y confirma las entregas. El **reponedor** usa una aplicación táctil instalable en el móvil para recorrer las máquinas y ajustar existencias. Y el **administrador de la tienda** tiene en el back-office de PrestaShop un menú completo con hoteles, precios, productos por establecimiento, pedidos, máquinas, liquidaciones y claves de API.

2. Compatibilidad

PrestaShop	1.7.8.0 a 9.x (declarado <code>min 1.7.8.0</code> , <code>max 9.99.99</code>)
PHP	7.4 a 8.3 — verificado con <code>php -l</code> sobre los 92 ficheros PHP en ambas ramas
Base de datos	MySQL 5.6+ / MariaDB 10.x. Usa <code>GET_LOCK</code> / <code>RELEASE_LOCK</code>
Dependencias	Ninguna. Cero Composer, cero <code>vendor/</code> , cero paquetes externos
Arquitectura	Legacy: clase en espacio de nombres global, <code>classes/</code> cargadas con <code>require_once</code> , controladores admin heredando de <code>ModuleAdminController</code> . Sin <code>src/</code> , sin <code>config/services.yml</code>
Idiomas	Castellano, inglés, francés, alemán, italiano, portugués y catalán (<code>translations/</code> , sistema nuevo XLIFF)
Pasarela de pago	Redsys (entorno de pruebas y real). También admite delegar en módulos de pago de terceros ya instalados
Multitienda	Ver la advertencia más abajo

Extensiones de PHP necesarias: `openssl` (para cifrar la clave de Redsys; sin ella la clave se guardaría en claro), `mbstring` , `json` y `curl` . El código QR se genera solo si la tienda dispone de la biblioteca correspondiente; si no, el módulo sigue funcionando y muestra la dirección en texto.

Multitienda — léase antes de instalar

El módulo es **consciente** de multitienda a nivel de datos: la ficha de hotel guarda su `id_shop` y todas las consultas de catálogo, nombres y precios filtran por tienda.

Pero **toda la configuración es global**, no por tienda. Cada opción de la pantalla de ajustes se guarda con `Configuration::getGlobalValue` / `updateGlobalValue` : las credenciales de Redsys, el remitente de los correos, los umbrales de las máquinas, los intentos de acceso al panel, la depuración. Si se cambia un valor estando en el contexto de una tienda concreta, **cambia para todas**. Esto no está avisado en la propia pantalla, así que conviene tenerlo presente antes de trabajar en una instalación con varias tiendas.

3. Instalación

1. Subir la carpeta `ecom_healthcorner` a `/modules/` de la tienda, o instalar el ZIP desde **Módulos** → **Gestor de módulos** → **Subir un módulo**.
2. Pulsar **Instalar**.
3. Entrar en la configuración del módulo al menos una vez. La primera entrada dispara `addnewfeatures()` , que aplica las migraciones de esquema pendientes y repara el menú.

La carpeta tiene que llamarse `ecom_healthcorner`. Todos los `require_once` internos y las rutas de plantilla apuntan a ese nombre en firme; si se renombra la carpeta, el módulo no arranca.

Qué crea al instalarse

Quince tablas con el prefijo de la tienda:

```
ecomhc_hotel , ecomhc_hotel_user , ecomhc_hotel_product , ecomhc_hotel_cms , ecomhc_price ,  
ecomhc_cart_meta , ecomhc_guest_order , ecomhc_payment , ecomhc_machine , ecomhc_machine_stock ,  
ecomhc_machine_movement , ecomhc_machine_user , ecomhc_api_key , ecomhc_panel_login ,  
ecomhc_venue_visit .
```

Un menú propio de primer nivel en el back-office, con estos submenús: Hoteles, Productos, Precios, Pedidos, Máquinas, Liquidaciones, API, Registros, Configuración y Ayuda.

Un grupo de clientes propio, para poder aplicar reglas de precio solo a los huéspedes.

Una zona y dos transportistas: uno para la entrega en el hotel y otro para la recogida en la máquina.

Cinco estados de pedido: «Preparando el pedido», «Listo para recoger», «Listo para recoger en máquina», «Entrega en tu hotel» y «Entregado al huésped».

Una estructura de páginas CMS: una categoría «Hoteles», dentro una subcategoría «Base Health Corner» y siete hojas legales base (condiciones de venta, política de privacidad, política de cookies, aviso legal, entregas y plazos, devoluciones y desistimiento, y qué es Health Corner). Al crear un hotel, si la opción está activada, estas hojas se duplican en una subcategoría suya sustituyendo los datos del establecimiento.

Dieciocho hooks declarados, entre ellos `moduleRoutes` , `paymentOptions` , `actionValidateOrder` , `displayAdminOrder` , `displayAdminProductsExtra` y `addWebserviceResources` .

Qué pasa al desinstalar

Los datos se conservan. El método `uninstall()` desactiva los dos transportistas (marcado lógico, para no romper los pedidos ya emitidos), retira el menú del back-office y llama al `uninstall()` del núcleo. **No borra ninguna tabla, ningún pedido, ningún hotel y ninguna página CMS.** Volver a instalar recupera todo tal cual estaba.

Existe un fichero `sql/uninstall.php` con las sentencias `DROP TABLE` , pero **el proceso de desinstalación no lo ejecuta:** está ahí para un borrado manual y deliberado. Conviene saberlo para no darlo por hecho en ninguna de las dos direcciones.

4. Configuración

Sobre las capturas de pantalla. Este manual no las lleva, y es deliberado. La entrega autorizada para este trabajo no incluía instalar el módulo en las tiendas PrestaShop locales, y sin una tienda con hoteles, productos y máquinas dados de alta las pantallas saldrían vacías o, peor, con datos inventados. Un manual que enseña una pantalla simulada como si fuera el producto engaña más de lo que ayuda.

Cuando se autorice la instalación en local, las capturas se generan y se incrustan en esta sección sin tocar el resto del texto.

La configuración está en un **único formulario con siete pestañas** y un solo botón de guardar. No hay siete formularios independientes: se rellena todo y se guarda una vez.

Pestaña «Pago (Redsys)»

Aquí se decide **quién cobra**. El interruptor *Activar pago con tarjeta (Redsys)* enciende la pasarela propia del módulo. Si se deja apagado, el checkout ofrece los módulos de pago que ya tenga instalados la tienda, y Health Corner solo se ocupa del pedido.

Con Redsys activo hay que rellenar el **FUC** (el código de comercio que da el banco), el **Terminal** (normalmente **1**) y la **Clave Redsys**. Esa clave **se guarda cifrada** en la base de datos con AES-256 y una firma HMAC; no se almacena en claro ni aparece en los registros. También se elige el entorno: pruebas mientras se valida la integración, y real cuando el banco da el visto bueno.

Lo importante de esta pestaña: el pedido **nunca se da por pagado desde el navegador del cliente**. Redsys avisa al servidor por su cuenta, el módulo comprueba la firma, comprueba que el importe cobrado coincide con el que se lanzó, y solo entonces crea el pedido. Si el cliente cierra el navegador después de pagar, el pedido se crea igual.

Pestaña «Checkout y emails»

El **remite de los correos** (nombre y dirección) es lo que verá el huésped en su bandeja de entrada. Conviene poner algo reconocible para el hotel, no la dirección genérica de la tienda, o el correo parecerá publicidad.

DNI obligatorio y Teléfono obligatorio añaden esos campos al formulario de compra. Pedir menos datos hace que se abandonen menos carritos; pedir el teléfono ayuda cuando hay que avisar de un pedido que no se recoge. El DNI solo tiene sentido si el tipo de producto lo exige.

Checkout de máquina decide si un producto que **solo** está en la máquina expendedora se puede comprar desde el móvil o hay que comprarlo en la propia máquina. Está apagado por defecto, y ese apagado se aplica **en el servidor**, no solo escondiendo el botón: aunque alguien manipule la página, el carrito rechaza el producto.

Pestaña «Panel de negocios»

Panel de hoteles enciende o apaga el acceso del personal del establecimiento. Los otros dos campos son la defensa contra quien intente adivinar contraseñas: *Intentos login panel* (cuántos fallos se toleran) y *Minutos bloqueo panel* (cuánto tiempo se cuentan). Los valores por omisión son 5 intentos y 15 minutos, y se aplican tanto por dirección IP como por correo electrónico. Bajar los intentos a 3 endurece el acceso; subirlos por encima de 10 lo deja prácticamente sin protección.

Pestaña «Máquinas y planograma»

Ajustes de cómo se dibuja el planograma, la rejilla que representa los canales de la máquina: tamaño de la imagen del producto, longitud del nombre y capacidad por canal. Son valores de presentación; se tocan

cuando la rejilla se ve apretada o los nombres salen cortados en la tablet del reponedor.

Pestaña «API»

Aquí se ajusta el **límite de peticiones por minuto y clave** (120 si se deja vacío). Es el tope que puede hacer una máquina expendedora o el software que la gobierna. Las claves en sí no se crean aquí sino en el submenú **API** del menú del módulo.

Pestaña «Aislamiento»

Controla que un huésped que entra en la tienda de un hotel no acabe navegando por el catálogo general de la tienda madre. Permite además declarar direcciones IP que se saltan el bloqueo, útil para que el propio administrador pueda mirar ambas cosas desde la oficina.

Pestaña «Avanzado»

Meses purga datos es el plazo de conservación de los datos personales del huésped. Pasado ese tiempo, la tarea programada los anonimiza: el pedido se conserva por obligación contable, pero sin nombre, sin habitación y sin contacto.

Copiar hojas legales al crear hotel automatiza la duplicación de las siete páginas legales para cada establecimiento nuevo.

Segundos de inactividad (quiosco) es para las tablets que se dejan fijas en recepción: pasado ese tiempo sin tocar la pantalla, la sesión se reinicia para que el siguiente huésped no vea el carrito del anterior.

Depuración (logs) activa el registro detallado en [logs/](#). Se enciende para diagnosticar y **se apaga después**: con mucho tráfico los ficheros crecen deprisa.

Kill switch envíos corta de golpe todos los avisos automáticos. Es el freno de mano: si algo va mal con los correos, se activa esto y deja de salir nada mientras se investiga.

5. Cómo se usa, día a día

Dar de alta un hotel

En **Health Corner** → **Hoteles** → **Añadir**. Se rellenan el nombre, la dirección web corta (el *slug*), el color de la marca, el logotipo, los horarios de entrega y cómo se llama la unidad de alojamiento —«habitación» en un hotel, pero puede ser «camarote», «apartamento» o «sala» según el negocio—.

Al guardar, el módulo genera la dirección de la tienda ([/hc/{slug}](#)), el código QR para imprimir, la subcategoría de páginas legales y el enlace al panel del hotel. En la ficha de detalle hay además **códigos QR con atribución por ubicación** —recepción, habitación, ascensor, spa, piscina, restaurante, gimnasio y máquina—: cada uno abre la misma tienda pero deja constancia de por dónde entró el huésped, de modo que al cabo de unas semanas se sabe qué cartel funciona y cuál no.

El logotipo se sube desde la misma ficha. El módulo comprueba que el fichero es de verdad una imagen, no solo que la extensión lo parezca, y lo reescala a 480×480 en PNG.

Poner productos y precios

En **Productos** se eligen, hotel por hotel, qué artículos del catálogo se venden, si van por entrega, por máquina o por ambos canales, en qué orden se muestran y cuáles son destacados. Hay importación por CSV para no hacerlo a mano cuando son muchos.

En **Precios** se fija cuánto cuesta cada producto en cada hotel. Hay tres modos: *heredar* el precio del catálogo general, aplicar un *porcentaje* sobre él, o poner un precio *fijo*. Lo habitual es heredar por omisión y usar el porcentaje para reflejar la comisión del establecimiento, dejando el precio fijo para casos concretos.

El día a día del hotel

El personal entra en `/hc/{slug}/panel` con el correo y la contraseña que se le hayan asignado. Esas credenciales son las de un cliente de PrestaShop al que se ha dado de alta como usuario del panel desde la ficha del hotel; al crearlo, el módulo genera una contraseña y **la muestra una sola vez** en pantalla. Hay que copiarla en ese momento.

Dentro ve los pedidos de su establecimiento, con el código de recogida de cada uno. Cuando entrega uno, lo marca como entregado tecleando o escaneando ese código. El panel solo deja ver y tocar los pedidos de su hotel.

El reponedor

En `/hc/reponedor` hay una aplicación pensada para el móvil, instalable como aplicación. El reponedor entra con sus credenciales y sigue un asistente: elige el negocio, elige la máquina y ajusta las existencias con cajas grandes de subir y bajar, con buscador para no perder tiempo. Toda escritura queda registrada como movimiento auditable, con quién y cuándo.

Puede entrar tanto quien esté dado de alta como reponedor —y si se le asigna el hotel `0`, accede a todos— como quien sea usuario del panel de un hotel, porque quien gestiona el establecimiento suele reponer también sus máquinas.

Liquidaciones

En **Liquidaciones** se saca lo que hay que pagarle a cada hotel por sus ventas, con exportación a CSV para pasárselo a contabilidad.

Conectar la máquina expendedora

En **API** se crea una clave por máquina o por sistema, con su etiqueta, el negocio al que se limita y si tiene permiso de escritura. **La clave se muestra una sola vez**: de ella solo se guarda una huella, no se puede volver a mostrar y si se pierde hay que revocarla y crear otra.

La máquina llama a `https://{dominio}/hc/api/v1/{recurso}` con la cabecera `X-HC-API-Key`. Los recursos disponibles son `ping`, `hotels`, `machines`, `products`, `stock/shop`, `stock/machine`, `movements` y `restock`. Las escrituras son idempotentes por el campo `reference`: reenviar el mismo movimiento no lo duplica, que es justo lo que hace falta cuando una máquina pierde la cobertura y reintenta.

Conviene dar permiso de escritura **solo** a la máquina que registra ventas. Para un panel que únicamente consulta, con lectura basta.

La tarea programada

Hay que llamar una vez al día, desde el cron del hosting, a:

```
https://{dominio}/hc/cron?token={TOKEN}&task=all
```

El token está en la pantalla de la API. Esa llamada hace dos cosas: anonimizar los datos personales que han cumplido el plazo de conservación y revisar las alertas de reposición. Sin ella, el módulo sigue vendiendo con normalidad, pero los datos personales se acumulan sin caducar y nadie avisa de que una máquina se está quedando vacía.

6. Qué ve el cliente en la tienda

El huésped escanea el QR y entra en `/hc/{slug}`. Lo que ve **no se parece a la tienda madre**: es una plantilla propia, con su cabecera, su logotipo y su color, y una hoja de estilo independiente. No hay menú de la tienda general, ni pie de página corporativo, ni enlaces que le saquen de ahí. Las páginas legales que ve son las de su hotel.

El recorrido es corto a propósito, porque casi siempre lo hace de pie y con una mano: catálogo con buscador → ficha de producto → carrito → datos (nombre, habitación y lo que se haya marcado como obligatorio) → pago con tarjeta → confirmación con el código de recogida y su código QR.

Después recibe un correo con la confirmación y puede seguir su pedido en `/hc/{slug}/track`, una página que muestra una línea de tiempo con cuatro pasos: pago confirmado, preparando el pedido, listo (para entregar o en la máquina, según corresponda) y entregado.

Los productos que solo están en la máquina se muestran marcados como tales, y con el checkout de máquina apagado no se pueden añadir al carrito: se indican para que el huésped sepa que existen y baje a por ellos.

En las tablets fijas de recepción funciona el **modo quiosco**: tras los segundos de inactividad configurados avisa y reinicia la sesión, para que nadie se encuentre el carrito del huésped anterior.

7. Preguntas frecuentes y problemas

El QR abre una página que no existe. Faltan las URL amigables. En **Parámetros de la tienda → Tráfico y SEO** hay que activar las URL simplificadas. Las rutas del módulo (`/hc/...`) las registra el hook `moduleRoutes` y sin esa opción no funcionan.

El pedido se pagó pero no aparece. Redsys avisa al servidor por su cuenta, a `/hc/notify`. Si esa dirección no es accesible desde fuera —un cortafuegos, una tienda en mantenimiento, un `.htaccess` restrictivo, una contraseña HTTP delante— el aviso no llega y el pedido no se crea aunque el cobro sí haya ocurrido. Es lo primero que hay que comprobar. En **Registros** queda constancia de cada aviso recibido y de por qué se rechazó, si se rechazó.

Los cambios de precio no se ven en la tienda del hotel. Es la caché de plantillas. Vaciar la caché de PrestaShop desde **Parámetros avanzados** → **Rendimiento**. Si persiste, revisar que el producto esté asignado a ese hotel en **Productos**: un precio sin asignación no se aplica.

El personal del hotel no puede entrar en el panel. Tres causas, por orden de frecuencia: el panel está apagado en la configuración; el cliente de PrestaShop existe pero no está dado de alta como usuario de ese hotel; o se han agotado los intentos y está bloqueado durante los minutos configurados. Los intentos fallidos se registran con su fecha, su IP y su correo.

Se ha perdido la clave de la API. No se puede recuperar: de la clave solo se guarda su huella. Hay que revocar la que se perdió y crear una nueva, y reconfigurar la máquina con ella.

Un huésped pide que borren sus datos. En la pantalla de la API hay una búsqueda por correo o por documento con dos botones: exportar todos sus datos (derecho de acceso) y anonimizar (derecho de supresión). Los pedidos se conservan sin datos personales, porque la obligación contable así lo exige.

El módulo no envía correos. Comprobar primero que el *kill switch* de la pestaña Avanzado esté apagado: si está encendido, corta todos los envíos a propósito y no hay ningún error en ningún sitio.

Los textos salen en castellano aunque la tienda esté en inglés. Es una limitación conocida y está explicada en la ficha técnica: buena parte de las cadenas del back-office todavía no están volcadas a los ficheros de traducción y el sistema devuelve el texto original. No afecta al recorrido de compra del huésped, que sí está traducido.

8. Seguridad

Se ha revisado el módulo entero: el fichero principal, las 31 clases de `classes/`, los 11 controladores de administración, los 13 controladores de front, las 26 plantillas y los 6 ficheros de JavaScript.

Lo que ya estaba bien

Conviene decirlo, porque no es lo habitual: **la base de este módulo es sólida.**

- **No hay inyección de SQL.** Todos los valores que llegan del usuario pasan por `pSQL()` o por una conversión a entero antes de entrar en una consulta. No se encontró ni un solo caso de interpolación directa.
- **La firma de Redsys se valida antes de tocar nada,** con `hash_equals` y normalizando los dos alfabetos base64 antes de comparar. Además se comprueba que el importe firmado por el banco coincide con el que se registró al lanzar el pago, hay bloqueo atómico por carrito con `GET_LOCK` para que dos avisos simultáneos no creen dos pedidos, y la operación es idempotente.
- **La clave de Redsys se guarda cifrada** con AES-256-CBC y HMAC-SHA256 (cifrar y luego firmar), operando los datos binarios con funciones de bytes y no con las de `mbstring`, que es el error clásico que rompe el descifrado.
- **Las claves de API se guardan solo como huella** y se comparan en tiempo constante.
- **El acceso al panel y a la aplicación del reponedor** está protegido con limitación de intentos por IP y por correo, comprobación de contraseña en tiempo constante también cuando el usuario no existe (para

que no se pueda averiguar qué correos están dados de alta midiendo el tiempo de respuesta), sesión propia firmada con HMAC, caducidad a las 8 horas y revalidación en vivo contra la base de datos en cada petición.

- **La autorización está comprobada de verdad:** cada acción del reponedor y del panel verifica que el hotel afectado está entre los suyos. Las acciones que modifican datos llevan su propio testigo anti-CSRF.
- **El código de recogida** tiene 8 caracteres de un alfabeto de 31, generados con `random_int`: unos 850.000 millones de combinaciones. No es adivinable.
- **La subida del logotipo** valida tamaño, extensión y que el fichero sea realmente una imagen, y lo reescala a PNG.
- **La lectura de los ficheros de registro** está acotada con `basename()` y exige el prefijo `ecomhc-`: no hay salto de directorio.
- **No hay `eval`, ni `exec`, ni `unserialize` de datos externos, ni `var_dump` olvidados, ni `nofilter` sin sanear, ni credenciales escritas en el código.** La carpeta `logs/` tiene su `.htaccess` que la cierra y su `index.php`; hay `index.php` en las 33 carpetas del módulo (la raíz y sus 32 subcarpetas).
- **El aviso de Redsys y la tarea programada no barren histórico:** no hay ningún bucle que recorra pedidos antiguos disparando correos.

Lo que se ha corregido en esta revisión

1. El CSS personalizado de cada hotel podía cargar recursos de dominios externos

(`classes/EcomHcCms.php`, método `sanitizeCss`).

Cada hotel tiene un campo de CSS *adicional* que se inyecta tal cual en un bloque `<style>` de todas sus páginas. El saneador bloqueaba `expression()`, `javascript:` y `data:text`, pero dejaba pasar cualquier `url()` apuntando fuera. Con una línea como `body{background:url(https://un-tercero.example/pixel.png)}` —o una `@font-face` con una tipografía alojada fuera— **el navegador de cada huésped pide ese recurso al tercero y le entrega su dirección IP, su navegador y la dirección de la página que está viendo**. Es una cesión de datos personales que no figura en la política de privacidad del hotel, en una tienda de productos de parafarmacia, donde el dato de qué se está mirando es especialmente delicado.

Ahora las `url()` se limitan a rutas de la propia tienda —relativas, o absolutas con el dominio de la tienda— y a imágenes embebidas `data:image` de formatos que no ejecutan código. Cualquier otra cosa se convierte en `none`. De paso se han cerrado tres huecos más del mismo filtro: los escapes de CSS (con los que se podía reconstruir `expression()` o `javascript:` letra a letra y burlar los filtros), los comentarios `/* */` en medio de una palabra clave, `-moz-binding`, `@charset` y `@namespace`, y las comillas dentro de una `url()` con las que se podían encadenar reglas propias. Se bloquea también `data:image/svg+xml`, porque un SVG puede llevar script dentro.

Comprobado con un banco de pruebas de 16 ataques y 6 casos de CSS legítimo: los 16 ataques quedan neutralizados y los 6 casos legítimos siguen funcionando. Efecto secundario aceptado: las secuencias de escape de CSS dejan de funcionar, así que un `content: "\201C"` mostrará `201C`. Es un precio muy pequeño por cerrar el filtro, y afecta a un uso raro en un campo que solo toca el administrador.

2. El buscador de productos del front no llevaba cabeceras anti-caché (`controllers/front/catalog.php`).

El desplegable de sugerencias devuelve catálogo, disponibilidad y precios —estado que cambia— y salía sin ninguna instrucción de no cachear. Un proxy intermedio o el propio navegador podía servir una respuesta

congelada, mostrando productos que ya no están o precios viejos. Se han añadido las tres cabeceras (`Cache-Control` , `Pragma` , `Expires`) y `cache: 'no-store'` en la llamada del JavaScript. No se ha tocado el `Content-Type` , que ya estaba y es correcto aquí.

3. La tarea programada tampoco las llevaba (`controllers/front/cron.php`). Mismas tres cabeceras. Una respuesta cacheada de un cron puede hacer creer que la tarea se ejecutó cuando no lo hizo.

4. Las dos búsquedas del back-office (clientes en la ficha de hotel, productos en la pantalla de productos) tampoco pedían `cache: 'no-store'` desde el navegador, aunque el servidor sí mandaba la cabecera. Ahora lo piden las dos partes.

5. La línea de tiempo del seguimiento estaba escrita en castellano fijo, sin traducir

(`classes/EcomHcPickup.php`). «Pago confirmado», «Preparando el pedido», «Listo en la máquina», «Listo para entregar» y «Entregado» estaban directamente en el código, de modo que un huésped con la tienda en inglés o en alemán las veía en castellano. Ahora pasan por el traductor, y se han añadido las cuatro que faltaban a los **siete idiomas** del módulo, traducidas. La quinta ya estaba.

6. Ortografía castellana: 231 correcciones en 32 ficheros. Textos que ya estaban delante del huésped se mostraban sin tildes ni eñes: «Anadir al carrito», «Numero de habitacion», «Telefono», «Acepto las condiciones y la politica de privacidad», «En unos segundos recibiras la confirmacion por email», «Teclea el codigo a mano», «Contrasena», «Sesion caducada», «En la maquina». Se han corregido las etiquetas, las descripciones de los campos de configuración, los textos de las plantillas, los mensajes del JavaScript y los destinos en castellano de los ficheros de traducción.

Tres cadenas se han dejado **a propósito sin tilde**: son claves de traducción, no texto visible. En PrestaShop la cadena que se escribe en el código es la llave con la que se busca en el fichero de traducción; ponerle una tilde la desconecta y el módulo pierde la traducción en todos los idiomas. El texto acentuado que ve el usuario está en el destino del fichero `es-ES` , que sí se ha corregido.

La comprobación se hizo con un guion que recorre las **510 claves de traducción** que usa el código y las compara con los ficheros: detecta las que llevan tilde pero cuya versión sin tilde sí existe como identificador, que son exactamente las que la corrección habría roto. Salieron tres, se revirtieron, y al repetir la comprobación **no queda ninguna**. Se verificó además que los **3.934 identificadores** de los ficheros de traducción siguen idénticos.

Comprobaciones hechas sobre los cambios

- `php -l` sobre los **92 ficheros PHP**, en **PHP 7.4 y PHP 8.3**: sin errores.
- Los **14 ficheros de traducción** siguen siendo XML válido.
- Los **6 ficheros de JavaScript** pasan `node --check` .
- Los identificadores de traducción (`resname` y `source`) son **idénticos byte a byte** a como estaban antes de la corrección ortográfica.
- Ninguna cadena en castellano se ha colado en los correos en portugués, italiano, francés, alemán, catalán o inglés.
- El tamaño de cada fichero tocado se comprobó antes y después: ninguno se ha truncado.

Lo que queda pendiente

1. Traducciones incompletas del back-office. De las 510 cadenas traducibles que usa el código, **271 no tienen entrada en los ficheros de traducción**. No es un fallo introducido en esta revisión, sino un hueco que ya estaba. Como PrestaShop devuelve el texto original cuando no encuentra la traducción, en castellano se ve bien y en los otros seis idiomas se ve castellano. Cerrarlo son 271 cadenas por 7 idiomas: es un trabajo aparte, no un arreglo.

2. La configuración es global, no por tienda. Descrito en el apartado 2. La pantalla no avisa de ello cuando la multitienda está activa. Añadir el aviso es sencillo; convertir cada opción en configurable por tienda cambiaría el comportamiento del módulo y no se ha tocado.

3. Las claves de las pestañas del formulario no llevan prefijo del módulo. Son `redsys`, `checkout`, `panel`, `maquinas`, `api`, `aislamiento` y `avanzado`. La norma interna pide prefijarlas, porque esas claves se convierten en identificadores del DOM y una colisión con un identificador que ya exista en el back-office saca esos campos fuera del formulario y hace que la pestaña deje de guardarse diciendo «guardado». **Se ha comprobado contra el código real de PrestaShop 8.2.7 y 9.1.1 que ninguna de las siete colisiona**, así que hoy no hay ningún problema. Queda anotado como deuda: si en una versión futura apareciera un `id="panel"` o un `id="api"` en el back-office, esa pestaña dejaría de guardar en silencio.

4. La página de seguimiento no tiene límite de intentos. Se puede pedir `/hc/{slug}/track?c=XXXXXXX` tantas veces como se quiera. Con 850.000 millones de combinaciones posibles y unos pocos miles de pedidos, adivinar uno es inviable, así que el riesgo real es despreciable; pero un límite por IP sería barato y evitaría el ruido en los registros.

5. La contraseña generada al crear un usuario del panel se muestra en pantalla en el back-office. Es deliberado —hay que poder entregársela a alguien— y solo la ve el empleado que crea el usuario, pero conviene no dejar esa pantalla abierta ni capturarla.

6. La API responde con `Access-Control-Allow-Origin: *`. Es correcto para lo que es —una API sin sesión, autenticada por cabecera, que no usa cookies—, así que un navegador ajeno no puede aprovecharlo. Se anota por transparencia, no como fallo.

9. Ficha técnica

Identificación

Nombre interno	<code>ecom_healthcorner</code>
Clase principal	<code>Ecom_Healthcorner</code> <code>extends</code> <code>PaymentModule</code> (espacio de nombres global)
Versión	1.0.0
Pestaña	<code>front_office_features</code>
Traducciones	Sistema nuevo. Dominios <code>Modules.Ecomhealthcorner.Shop</code> y <code>Modules.Ecomhealthcorner.Admin</code>

Tablas

Tabla	Contenido
<code>ecomhc_hotel</code>	Ficha del establecimiento: nombre, slug, color, logotipo, horarios, CSS propio
<code>ecomhc_hotel_user</code>	Qué clientes pueden entrar en el panel de qué hotel
<code>ecomhc_hotel_product</code>	Qué productos vende cada hotel y por qué canal
<code>ecomhc_hotel_cms</code>	Correspondencia hotel → página CMS por código
<code>ecomhc_price</code>	Precio por hotel y producto (heredado, porcentaje o fijo)
<code>ecomhc_cart_meta</code>	A qué hotel y canal pertenece cada carrito
<code>ecomhc_guest_order</code>	Datos del huésped y código de recogida del pedido
<code>ecomhc_payment</code>	Transacciones de Redsys: <code>ds_order</code> , importe, respuesta, token
<code>ecomhc_machine</code>	Máquinas expendedoras
<code>ecomhc_machine_stock</code>	Existencias por máquina y canal
<code>ecomhc_machine_movement</code>	Registro auditable de cada movimiento
<code>ecomhc_machine_user</code>	Altas de reponedores (hotel <code>0</code> = todos)
<code>ecomhc_api_key</code>	Claves de API: huella, permisos, uso
<code>ecomhc_panel_login</code>	Intentos de acceso al panel, para la limitación
<code>ecomhc_venue_visit</code>	Visitas por establecimiento y origen del QR

Hooks

`moduleRoutes`, `actionGetProductPropertiesAfter`, `actionCartUpdateQuantityBefore`, `actionValidateOrder`, `actionPaymentConfirmation`, `paymentOptions`, `actionCarrierProcess`, `actionAuthentication`, `actionCustomerLogoutAfter`, `displayHeader`, `displayAdminProductsExtra`, `actionProductSave`, `actionAdminControllerSetMedia`, `actionAdminMetaBeforeWriteRobotsFile`, `addWebserviceResources`, `displayAdminOrder`, `displayAdminOrderSide`. Más `actionProductPriceCalculation` según la versión.

Controladores de administración

`AdminEcomHc` (raíz del menú), `AdminEcomHcHotels`, `AdminEcomHcProducts`, `AdminEcomHcPrices`, `AdminEcomHcOrders`, `AdminEcomHcMachines`, `AdminEcomHcSettlements`, `AdminEcomHcApi`, `AdminEcomHcLogs`, `AdminEcomHcConfig`, `AdminEcomHcHelp`.

Direcciones del front

Ruta amigable	Para qué
<code>/hc/{slug}</code>	Catálogo del establecimiento
<code>/hc/{slug}/product/...</code>	Ficha de producto
<code>/hc/{slug}/cart</code>	Carrito (y sus llamadas AJAX)
<code>/hc/{slug}/checkout</code>	Datos del huésped
<code>/hc/{slug}/payment</code>	Salto a la pasarela
<code>/hc/{slug}/confirmation</code>	Confirmación de vuelta del banco
<code>/hc/{slug}/track?c=CODIGO</code>	Seguimiento del pedido
<code>/hc/{slug}/legal/{code}</code>	Hoja legal del establecimiento
<code>/hc/{slug}/panel</code>	Panel del personal del hotel
<code>/hc/reponedor</code>	Aplicación del reponedor
<code>/hc/api/v1/{recurso}</code>	API para las máquinas
<code>/hc/notify</code>	Aviso servidor a servidor de Redsys
<code>/hc/cron?token=...&task=all</code>	Tarea programada

API

Autenticación por cabecera `X-HC-API-Key`. Exige HTTPS salvo en modo desarrollo. Límite por omisión de 120 peticiones por minuto y clave. Recursos: `ping`, `hotels`, `machines`, `products`, `stock/shop`, `stock/machine`, `movements`, `restock`. Escrituras idempotentes por `reference`. Documentación completa en `docs/04-API.md`.

Tarea programada

GET /hc/cron?token={ECOMHC_CRON_TOKEN}&task=rgpd|alerts|all , una vez al día. El testigo se compara con hash_equals . Sin él responde 403.

Registros

En logs/ dentro de la carpeta del módulo, protegida con .htaccess (Require all denied) e index.php . Se ven y se descargan desde **Health Corner** → **Registros**. Solo se registra en detalle si *Depuración* está activada, salvo un registro de diagnóstico de la ficha de hotel que escribe siempre en logs/diag.log .

Ajustes guardados

52 claves con el prefijo ECOMHC_ , todas globales. Las principales: ECOMHC_REDSYS_* (pasarela; la clave va cifrada), ECOMHC_PANEL_* (panel y limitación de intentos), ECOMHC_MACHINE_* (máquinas), ECOMHC_PLANOGRAM_* (planograma), ECOMHC_ORDER_STATE_* (estados creados), ECOMHC_CARRIER_* (transportistas), ECOMHC_CMS_* (páginas legales), ECOMHC_PURGE_MONTHS (conservación de datos), ECOMHC_CRON_TOKEN , ECOMHC_API_RATE , ECOMHC_DEBUG y ECOMHC_KILL_SWITCH .

Permisos de ficheros

Escritura en logs/ y en views/img/hotels/ (logotipos de los establecimientos). El resto, solo lectura.

Documentación técnica adicional

Dentro del módulo, en docs/ : 01-TECNICO.md , 02-DESARROLLO.md , 03-USUARIO.md , 04-API.md y CONTRATO_TECNICO.md .

Ecom Experts · ecomyseo@gmail.com · 2026 · Licencia AFL-3.0